



PDF Download
1569901.1570064.pdf
05 April 2026
Total Citations: 11
Total Downloads: 407

 Latest updates: <https://dl.acm.org/doi/10.1145/1569901.1570064>

RESEARCH-ARTICLE

Uncertainty handling CMA-ES for reinforcement learning

VERENA HEIDRICH-MEISNER, Ruhr University Bochum, Bochum, Nordrhein-Westfalen, Germany

CHRISTIAN IGEL, Ruhr University Bochum, Bochum, Nordrhein-Westfalen, Germany

Open Access Support provided by:

Ruhr University Bochum

Published: 08 July 2009

Citation in BibTeX format

GECCO'09: Genetic and Evolutionary
Computation Conference
July 8 - 12, 2009
Québec, Montreal, Canada

Conference Sponsors:
SIGEVO

Uncertainty Handling CMA-ES for Reinforcement Learning

Verena Heidrich-Meisner
Institut für Neuroinformatik
Ruhr-Universität Bochum
44780 Bochum, Germany
verena.heidrich-meisner
@neuroinformatik.rub.de

Christian Igel
Institut für Neuroinformatik
Ruhr-Universität Bochum
44780 Bochum, Germany
christian.igel@neuroinformatik.rub.de

ABSTRACT

The covariance matrix adaptation evolution strategy (CMA-ES) has proven to be a powerful method for reinforcement learning (RL). Recently, the CMA-ES has been augmented with an adaptive uncertainty handling mechanism. Because uncertainty is a typical property of RL problems this new algorithm, termed UH-CMA-ES, is promising for RL. The UH-CMA-ES dynamically adjusts the number of episodes considered in each evaluation of a policy. It controls the signal to noise ratio such that it is just high enough for a sufficiently good ranking of candidate policies, which in turn allows the evolutionary learning to find better solutions. This significantly increases the learning speed as well as the robustness without impairing the quality of the final solutions. We evaluate the UH-CMA-ES on fully and partially observable Markov decision processes with random start states and noisy observations. A canonical natural policy gradient method and random search serve as a baseline for comparison.

Categories and Subject Descriptors

I.2.6 [Computing Methodologies]: Artificial Intelligence—Learning; I.2.8 [Computing Methodologies]: Artificial Intelligence—Problem Solving, Control Methods, and Search

General Terms

Algorithms, Performance

Keywords

reinforcement learning, direct policy search, covariance matrix adaptation evolution strategy, uncertainty handling

1. INTRODUCTION

Evolution strategies have proven to be powerful methods for reinforcement learning (e.g., see [15, 19, 24, 7, 14, 12]). It has been argued that they are more robust against noise

and uncertainty than alternative approaches because they rely on ranking policies [14], and not on estimating absolute performance predictions or even their gradients. This is an important feature, because uncertainty and randomness arise from several sources in RL. The initial state usually varies, state-transitions and reward signals can be stochastic, and the state observations may be noisy. Further increasing the ability of evolutionary RL to cope with noise and uncertainty is the goal of this study.

Evolutionary algorithms for RL search directly in a parametrized policy space. The search direction can be determined by using estimates of the performance values or rankings of behavioral policies or by using estimates of performance gradients.

Uncertainty and randomness require that these estimates are based on a sample of several episodes (roll-outs). The sample size is a crucial parameter. If too few episodes are considered, the estimates are not reliable enough to allow for learning. If too many episodes are considered, the learning process becomes too slow. Unfortunately, it is usually not possible to determine an appropriate sample size for a given problem a priori (in practice we just make it “large enough”) and the optimal number may vary in the course of learning.

In this study, we consider the covariance matrix evolution strategy (CMA-ES, [11, 8, 27]) for direct policy search, which gives striking results on RL benchmark problems [15, 7, 14, 12]. The CMA-ES adapts the policy as well as parameters of its own search strategy (such as a variable metric) based on ranking policies. Albeit ranking is already much less susceptible to noise and uncertainty than estimating absolute performances or performance gradients, the ranking must be sufficiently accurate to evolve better policies. The reliability of the ranking depends on the degree of uncertainty as well as the number of roll-outs considered per performance estimation of each candidate solution. Recently the CMA-ES was augmented with an adaptive uncertainty handling scheme [9, 10], resulting in an algorithm termed UH-CMA-ES. The UH-CMA-ES has already been used successfully in model selection for supervised learning [6].

Based on [13], we suggest to apply the UH-CMA-ES to RL. The idea is that the uncertainty handling dynamically adapts the number of episodes for evaluating a policy such that the ranking of new candidate solutions is just reliable enough to drive the learning process. The uncertainty handling scheme is in principle independent of the CMA-ES and could be combined with other RL approaches and strategies for distributing evaluations among candidate solutions (e.g., for evolutionary online RL [28]).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

GECCO’09, July 8–12, 2009, Montréal Québec, Canada.
Copyright 2009 ACM 978-1-60558-325-9/09/07 ...\$5.00.

In the next section, we introduce the CMA-ES for direct policy search. In section 3, we describe the uncertainty handling. The empirical evaluation follows in section 4 before the conclusions.

2. BACKGROUND

We first briefly describe the general RL problem scenario and then the standard CMA-ES.

2.1 Reinforcement Learning

Markov decision processes (MDP) are the basic formalism to describe RL tasks. An MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$ consists of the set of states $\mathcal{S}$, the possible actions $\mathcal{A}$, the probabilities $\mathcal{P}_{s,s'}$ that an action $a \in \mathcal{A}$ taken in state $s \in \mathcal{S}$ leads to state $s' \in \mathcal{S}$, and the expected rewards $\mathcal{R}_{s,s'}^a$ received when going from state $s \in \mathcal{S}$ to $s' \in \mathcal{S}$ after performing action $a \in \mathcal{A}$.

Partially observable Markov decision processes (POMDP) are a generalization of MDPs [17]. In a POMDP, the environment is determined by an MDP, but the agent cannot directly observe the state of the MPD. Formally, a POMDP can be described as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, O \rangle$. The first four elements define the underlying MDP, Ω is the set of observations an agent can perceive, and the observation function $O : \mathcal{S} \times \mathcal{A} \rightarrow \Lambda(\Omega)$ maps a state and the action that resulted in this state to a probability distribution over observations (i.e., $O(s', a)(o)$ is the probability of observing $o \in O$ given that the agent took action $a \in \mathcal{A}$ and the new state is $s' \in \mathcal{S}$).

The goal of RL is to find a behavioral policy π such that some notion of expected future reward $\rho(\pi)$ is maximized. In this study, we just consider episodic tasks. That is, there are always final time steps, in the sense that the agent-environment interaction breaks naturally into subsequences or episodes. For an MDP, we can then define for example

$$\rho(\pi) = \sum_{s, s' \in \mathcal{S}, a \in \mathcal{A}} d^\pi(s) \pi(s, a) \mathcal{P}_{s, s'}^a \mathcal{R}_{s, s'}^a,$$

where $d^\pi(s) = \sum_{t=0}^{\infty} \gamma^t \Pr\{s_t = s | s_0, \pi\}$ is the stationary state distribution, which we assume to exist, s_t is the state in time step t , and $\gamma \in [0, 1]$ a discount parameter. The immediate reward received after the action a_t in time step t is denoted by $r_{t+1} \in \mathbb{R}$.

2.2 CMA-ES for Direct Policy Search

We employ the CMA-ES [11, 8, 27] for solving (PO)MPDPs. The highly efficient use of information and the fast adaptation of the mutation distribution make the CMA-ES one of the best direct evolutionary algorithms for real-valued optimization [3]. Here, the CMA-ES is used for direct policy search in a parametrized space of functions. In the following, we review the CMA-ES algorithm to which the uncertainty handling is added [9, 10] and its application to RL.

Covariance matrix adaptation evolution strategy.

In each generation k of the CMA-ES, which is shown in Algorithm 1, the l th offspring $\mathbf{x}_l^{(k+1)} \in \mathbb{R}^n$, $l \in \{1, \dots, \lambda\}$, is generated by multi-variate *Gaussian mutation* and *weighted global intermediate recombination*, that is,

$$\mathbf{x}_l^{(k+1)} \leftarrow \mathbf{m}^{(k)} + \sigma^{(k)} \mathbf{z}_l^{(k)},$$

where $\mathbf{z}_l^{(k)} \sim \mathcal{N}(\mathbf{0}, \mathbf{C}^{(k)})$ and $\mathbf{m}^{(k)} \leftarrow \sum_{l=1}^{\mu} w_l \mathbf{x}_{l:\lambda}^{(k)}$ with $\mathbf{x}_{l:\lambda}^{(k)}$ denoting the l th best individual of the λ offspring.

Algorithm 1: rank- μ CMA-ES

```

1 initialize  $\mathbf{m}^{(0)} = \boldsymbol{\theta}_{\text{init}}$ ,  $\sigma^{(0)}$ ,  $\mathbf{p}_\sigma^{(0)} = \mathbf{p}_c^{(0)} = \mathbf{0}$ ,  $\mathbf{C}^{(0)} = \mathbf{I}$ ,
    $n_{\text{eval}}^{(0)} = 1$ 
   //  $k$  counts number of generations / policy updates:
2 for  $k = 0, \dots$  do
3   for  $l = 1, \dots, \lambda$  do  $\mathbf{x}_l^{(k+1)} \sim \mathcal{N}(\mathbf{m}^{(k)}, \sigma^{(k)^2} \mathbf{C}^{(k)})$ 
   // evaluate offspring based on  $n_{\text{eval}}^{(k)}$  roll-outs
4   for  $l = 1, \dots, \lambda$  do  $f_l \leftarrow \text{performance}(\mathbf{x}_l^{(k+1)}, n_{\text{eval}}^{(k)})$ 
   // optional uncertainty handling, see section 3
5    $n_{\text{eval}}^{(k+1)} \leftarrow \text{uncertaintyHandling}$ 
   ( $\{\mathbf{x}_l^{(k+1)} | l \in \{1, \dots, \lambda\}\}$ )  $\mathbf{m}^{(k+1)} \leftarrow \sum_{i=1}^{\mu} w_i \mathbf{x}_{i:\lambda}^{(k)}$ 
6    $\mathbf{p}_\sigma^{(k+1)} \leftarrow$ 
    $(1 - c_\sigma) \mathbf{p}_\sigma^{(k)} + \sqrt{c_\sigma(2 - c_\sigma) \mu_{\text{eff}}} \mathbf{C}^{(k) - \frac{1}{2}} \frac{\mathbf{m}^{(k+1)} - \mathbf{m}^{(k)}}{\sigma^{(k)}}$ 
    $\sigma^{(k+1)} \leftarrow \sigma^{(k)} \exp \frac{c_\sigma}{d_\sigma} \left( \frac{\|\mathbf{p}_\sigma^{(k+1)}\|}{E\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1 \right)$ 
7    $\mathbf{p}_c^{(k+1)} \leftarrow (1 - c_c) \mathbf{p}_c^{(k)} + \sqrt{c_c(2 - c_c) \mu_{\text{eff}}} \frac{\mathbf{m}^{(k+1)} - \mathbf{m}^{(k)}}{\sigma^{(k)}}$ 
8    $\mathbf{C}^{(k+1)} \leftarrow (1 - c_{\text{cov}}) \mathbf{C}^{(k)} + \frac{c_{\text{cov}}}{\mu_{\text{cov}}} \mathbf{p}_c^{(k+1)} \mathbf{p}_c^{(k+1)T} +$ 
    $c_{\text{cov}} \left( 1 - \frac{1}{\mu_{\text{cov}}} \right) \sum_{i=1}^{\mu} w_i \mathbf{z}_{i:\lambda}^{(k)} \mathbf{z}_{i:\lambda}^{(k)T}$ 

```

This corresponds to non-elitist, rank-based selection. A common choice for the recombination weights is $w_l \propto \ln(\mu + 1) - \ln(l)$, $\|\mathbf{w}\|_1 = 1$, $\mathbf{w} \in \mathbb{R}^\mu$.

The CMA-ES is a variable metric algorithm adapting both the n -dimensional covariance matrix $\mathbf{C}^{(k)}$ of the normal mutation distribution as well as the *global step size* $\sigma^{(k)} \in \mathbb{R}^+$. The covariance matrix update has two parts, the rank-1 update considering the change of the population mean over time and the rank- μ update considering the successful variations in the last generation. The rank-1 update is based on a low-pass filtered *evolution path* $\mathbf{p}^{(k)}$ of selected steps

$$\mathbf{p}_c^{(k+1)} \leftarrow (1 - c_c) \mathbf{p}_c^{(k)} + \sqrt{c_c(2 - c_c) \mu_{\text{eff}}} \frac{\mathbf{m}^{(k+1)} - \mathbf{m}^{(k)}}{\sigma^{(k)}}$$

and aims at changing $\mathbf{C}^{(k)}$ to make steps in the promising direction $\mathbf{p}^{(k+1)}$ more likely by morphing the covariance towards $\left[\mathbf{p}_c^{(k+1)} \right] \left[\mathbf{p}_c^{(k+1)} \right]^T$. The backward time horizon of the cumulation process is approximately c_c^{-1} , where $c_c = 4/(n + 4)$. The *variance effective selection mass* $\mu_{\text{eff}} = (\sum_{l=1}^{\mu} w_l^2)^{-1}$ is a normalization constant. The rank- μ update aims at making the single steps that were selected in the last iteration more likely by morphing $\mathbf{C}^{(k)}$ towards the successful $\left[\mathbf{z}_{i:\lambda}^{(k)} \right] \left[\mathbf{z}_{i:\lambda}^{(k)} \right]^T$. Putting both updates together, we have

$$\begin{aligned} \mathbf{C}^{(k+1)} \leftarrow & (1 - c_{\text{cov}}) \mathbf{C}^{(k)} + \frac{c_{\text{cov}}}{\mu_{\text{cov}}} \mathbf{p}_c^{(k+1)} \mathbf{p}_c^{(k+1)T} \\ & + c_{\text{cov}} \left(1 - \frac{1}{\mu_{\text{cov}}} \right) \sum_{i=1}^{\mu} w_i \mathbf{z}_{i:\lambda}^{(k)} \mathbf{z}_{i:\lambda}^{(k)T}. \end{aligned}$$

The learning rate of the covariance matrix update is set to $c_{\text{cov}} = \frac{2}{(n + \sqrt{2})^2}$. The parameter μ_{cov} mediates between the rank- μ and the rank-1 update and has default value $\mu_{\text{cov}} = \mu_{\text{eff}}$.

The global step size $\sigma^{(k)}$ is adapted on a faster timescale. It is increased if the selected steps are larger and/or more

correlated than expected and decreased if they are smaller and/or more anticorrelated than expected:

$$\sigma^{(k+1)} \leftarrow \sigma^{(k)} \exp \left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma^{(k+1)}\|}{\mathbb{E}\{\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|\}} - 1 \right) \right),$$

where the (*conjugate*) evolution path is

$$\begin{aligned} \mathbf{p}_\sigma^{(k+1)} &\leftarrow (1 - c_\sigma) \mathbf{p}_\sigma^{(k)} \\ &+ \sqrt{c_\sigma(2 - c_\sigma)\mu_{\text{eff}}} \mathbf{C}^{(k)-\frac{1}{2}} \frac{\mathbf{m}^{(k+1)} - \mathbf{m}^{(k)}}{\sigma^{(k)}}. \end{aligned}$$

Again, $c_\sigma = \frac{\mu_{\text{eff}} + 2}{n + \mu_{\text{eff}} + 3}$ is a fixed learning rate and $d_\sigma = 1 + 2 \max \left(0, \sqrt{\frac{\mu_{\text{eff}} - 1}{n + 1}} \right) + c_\sigma$ is a damping factor. The matrix $\mathbf{C}^{-\frac{1}{2}}$ is defined as $\mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T$, where $\mathbf{B}\mathbf{D}^2\mathbf{B}^T$ is an eigen-decomposition of $\mathbf{C}$ ($\mathbf{B}$ is an orthogonal matrix with the eigenvectors of $\mathbf{C}$ and $\mathbf{D}$ a diagonal matrix with the corresponding eigenvalues).

In our RL scenario, the function `performance`($\mathbf{x}$, $n_{\text{eval}}^{(k)}$) in Algorithm 1 corresponds to the evaluation of the policy with parameters $\boldsymbol{\theta} = \mathbf{x}$. The parameter $n_{\text{eval}}^{(k)}$ determines over how many episodes a performance (here the return, i.e., accumulated reward) average is computed. The uncertainty handling adapting the parameter $n_{\text{eval}}^{(k)}$ will be described in section 3.

The values of the learning rates and the damping factor are well considered and need not be adjusted dependent on the problem. Also the population sizes can be set to default values, which are $\lambda = \max(4 + \lfloor 3 \ln n \rfloor, 5)$ and $\mu = \lfloor \frac{n}{2} \rfloor$ for offspring and parent population, respectively [8]. If we fix $\mathbf{C}^{(0)} = \mathbf{I}$ and ignore the sample size $n_{\text{eval}}^{(k)}$, the only hyperparameter to be chosen problem dependent is the initial global step size $\sigma^{(0)}$. We use the CMA-ES implementation of the Shark machine learning library [16].

CMA-ES and direct policy search.

The CMA-ES was proposed for RL in [15]. By memorizing successful steps in the evolution path, the CMA-ES infers a search direction from scalar reward signals. In a more recent study, the CMA-ES (without rank- μ update) was compared to 8–12 (depending on the task) other RL algorithms including value-function and policy gradient approaches [7]. On the four test problems where the CMA-ES was considered, it ranked first, second (twice), and third. Further examples of successful applications of the CMA-ES for RL can be found in [19, 24] and additional comparisons on RL benchmarks in [12].

The CMA-ES for RL bears similarities to policy gradient methods (PGMs) [12]. Both search directly in policy space, but ESs for RL are actor-only methods while PGMs often have actor-critic architectures. In contrast to the CMA-ES, PGMs require a differentiable structure on the search space and stochastic policies for learning. Exploration of the search space is realized by random perturbations in both ESs and PGMs. Evolutionary methods usually perturb a deterministic policy by mutation and recombination, while in PGMs the random variations are an inherent property of the stochastic policies. In ESs there is one initial stochastic variation phase per generation. In contrast, the stochastic policy introduces perturbations in every time step of every episode. While the number n of parameters of the policy

Procedure uncertaintyHandling($\{\mathbf{x}_l | l = 1, \dots, \lambda\}$)

```
// reevaluate  $\lambda_{\text{reev}}$  solutions (w.l.o.g. the first  $\lambda_{\text{reev}}$ ):
1 for  $l = 1, \dots, \lambda_{\text{reev}}$  do  $f_l^{\text{reeval}} \leftarrow \text{performance}(\mathbf{x}_l, n_{\text{eval}})$ 
2 for  $l = \lambda_{\text{reev}} + 1, \dots, \lambda$  do  $f_l^{\text{reeval}} \leftarrow f_l$ 
// construct joint performance sample:
3  $\mathcal{L} \leftarrow ((\mathbf{x}_1, f_1, f_1), \dots, (\mathbf{x}_\lambda, f_\lambda, f_\lambda),$ 
    $(\mathbf{x}_1, f_1, f_1^{\text{reeval}}), \dots, (\mathbf{x}_\lambda, f_\lambda, f_\lambda^{\text{reeval}}))$ 
4 for  $l = 1, \dots, \lambda_{\text{reev}}$  do
   // compute rank change:
5    $\Delta_l \leftarrow |\text{rank}_{\mathcal{L}}^{\text{reeval}}(l) - \text{rank}_{\mathcal{L}}(l)| - 1$ 
   // determine uncertainty level:
6    $s \leftarrow \frac{1}{\lambda_{\text{reev}}} \sum_{i=1}^{\lambda_{\text{reev}}} (2\Delta_i - \Delta_{\theta}^{\text{lim}}(\text{rank}_{\mathcal{L}}^{\text{reeval}}(i) - \mathbb{I}\{f_i^{\text{reeval}} >$ 
    $f_i\}) - \Delta_{\theta}^{\text{lim}}(\text{rank}_{\mathcal{L}}(i) - \mathbb{I}\{f_i > f_i^{\text{reeval}}\}))$ 
// adjust number of evaluations:
7 if  $s > 0$  then  $n_{\text{eval}} \leftarrow \alpha n_{\text{eval}}$  else  $n_{\text{eval}} \leftarrow \frac{1}{\alpha} n_{\text{eval}}$ 
// update fitness values:
8 for  $l = 1, \dots, \lambda_{\text{reev}}$  do  $f_l \leftarrow \frac{f_l + f_l^{\text{reeval}}}{2}$ 
9 return  $n_{\text{eval}}$ 
```

determines the n -dimensional random variation in ES, in PGMs the usually lower dimensionality of the action corresponds to the dimensionality of the random perturbations. In ESs the search is driven solely by ranking policies and not by the absolute values of performance estimates or even their gradients. The reduced number of random events and the rank-based evaluation are decisive differences and we hypothesize that they allow ESs to be more robust.

The adaptation of the covariance matrix in the CMA-ES is similar to learning the (Fisher) metric in natural PGMs [18, 22, 20]. Arguably one of the most elegant state-of-the-art natural PGMs is the episodic natural actor-critic algorithm (NAC, [22, 20]), which serves as a baseline for comparison in our experiments.

3. UNCERTAINTY HANDLING FOR RANKING POLICIES

Evolutionary algorithms are well suited for optimization in noisy environments [2]. The population-based approach, the averaging in the recombination process, and the rank-based, non-elitist selection are features that make the CMA-ES less vulnerable to noise. However, if the signal to noise ratio is too small, special uncertainty handling is required. Here we use a slightly simplified version of the uncertainty handling heuristics proposed in [9]. It is called *UH-CMA-ES* and relies on adaptive reevaluation of solutions. The method is implemented in Procedure `uncertaintyHandling`. Because the selection process is rank-based, we only care about noise if it changes the ranking of offspring. In our scenario, individuals can be reevaluated and computing the mean or median of several evaluations reduces the level of uncertainty. However, the signal to noise ratio changes in the course of learning. Because every fitness evaluation is time consuming, we implement a strategy that adapts the number of evaluations per individual such that individuals are not evaluated too often, but still often enough that the fitness values can guide the optimization.

We use an algorithm to detect the effective noise by mon-

itoring the stability of the offspring ranking. Following [9], we construct a multi-set $\mathcal{L}$ containing 2λ triples of individuals with two corresponding sampled performance values. Let $f_i, i \in \{1, \dots, \lambda\}$, be the sampled performance of candidate solution $\mathbf{x}_i$. We reevaluate λ_{reev} candidate solutions and define f_i^{reeval} to be the newly obtained performance estimate if individual i was reevaluated and $f_i^{\text{reeval}} = f_i$ otherwise. We define

$$\mathcal{L} \leftarrow ((\mathbf{x}_1, f_1, f_1), \dots, (\mathbf{x}_\lambda, f_\lambda, f_\lambda), (\mathbf{x}_1, f_1, f_1^{\text{reeval}}), \dots, (\mathbf{x}_\lambda, f_\lambda, f_\lambda^{\text{reeval}})) .$$

That is, each policy parameter vector occurs twice in $\mathcal{L}$. In one of the corresponding triples the third component is the new performance estimate (which is always the same as the original estimate for all but λ_{reev} individuals). In the other triple the third component is the same as the second one (i.e., the original performance estimate).

Then we sort $\mathcal{L}$ twice using the first and second performance estimate (the second and the third component of the triples), respectively, and determine the corresponding ranks $\text{rank}_{\mathcal{L}}(i)$ and $\text{rank}_{\mathcal{L}}^{\text{reeval}}(i)$, respectively, of each reevaluated individual $\mathbf{x}_i$. Now we compute the *rank change*

$$\Delta_i \leftarrow |\text{rank}_{\mathcal{L}}^{\text{reeval}}(i) - \text{rank}_{\mathcal{L}}(i)| - 1 .$$

Considering the duplicated population $\mathcal{L}$ for ranking at first appears to be cumbersome and overly complex. However, by doing so it is ensured that the rank change is symmetric in the sense that it does not depend on the order in which the different performance estimates are obtained (e.g., sampled performance estimates such as $f_1 = 16, f_2 = 20, f_1^{\text{reeval}} = 12, f_2^{\text{reeval}} = 14$ lead to the same results as $f_1 = 16, f_2 = 14, f_1^{\text{reeval}} = 12, f_2^{\text{reeval}} = 20$).

The *uncertainty level* s is now defined by

$$s \leftarrow \frac{1}{\lambda_{\text{reev}}} \sum_{i, \mathbf{x}_i \text{ reevaluated}} (2\Delta_i - \Delta_{\theta}^{\text{lim}}(\text{rank}_{\mathcal{L}}^{\text{reeval}}(i) - \mathbb{I}\{f_i^{\text{reeval}} > f_i\}) - \Delta_{\theta}^{\text{lim}}(\text{rank}_{\mathcal{L}}(i) - \mathbb{I}\{f_i > f_i^{\text{reeval}}\})) .$$

The indicator function $\mathbb{I}$ is one if its argument is true and zero otherwise. The parameter $\theta \in [0, 1]$ (in our experiments set to 0.5 for the swimmer tasks and to 0.2 in all other cases) controls the level of noise we tolerate and $\Delta_{\theta}^{\text{lim}}(r)$ denotes the $\theta \times 50$ percentile of the possible rank changes (given by the $2\lambda - 1$ values $|1 - r|, |2 - r|, \dots, |2\lambda - 1 - r|$) when having the original rank r .

If $s > 0$ we increase the number of evaluations in the computation of a fitness value by a factor of α . Otherwise we decrease the number of evaluations by $1/\alpha$. We set $\alpha = 1.5$ in our experiments.

The reevaluation is done before the environmental selection in the standard CMA-ES, which then uses the median of the fitness values of the reevaluated individuals for ranking. The additional fitness evaluations increase the computational costs per generation. However, we reevaluate on average only $\bar{\lambda}_{\text{reev}} = \max(\lambda/10, 2)$ individuals in each generation.

Related work.

Schmidt, Branke and Chick [23] have developed a method for improving the accuracy in rank-based evolution strategies by using a ranking and selection procedure based on

Bayesian statistics. *Optimal computing budget allocation OCBA* [4] greedily allocates additional evaluations to individuals where this promises the largest improvement. The probability that the individual with the observed best mean is indeed the best individual within some indifference zone is maximized by adding more evaluations. Each individual has its own budget of samples. Sampling is stopped when a predefined level of probability of “good” selection is reached. Schmidt et al. apply *OCBA* to sets of comparisons between individuals necessary for obtaining rankings for different evolution strategies, thus effectively reevaluating only those individuals that are helpful to improve the accuracy of the ranking of the considered evolution strategy.

That all offspring in one generation have the same budget of roll-outs is a drawback of our approach. However, in the non-elitist CMA-ES, in which all offspring are generated from the weighted center of mass of the parent population, individual strategy parameters cannot directly be implemented.

4. EXPERIMENTS

We first describe the benchmark problems, then the algorithms we consider for comparison, after that the precise experimental setup, and finally the results.

4.1 Benchmark problems

We consider several RL benchmarks taken from the literature to investigate the influence of our uncertainty handling methods.

Mountain car and noisy mountain car.

In the mountain car task an underpowered car is to be driven out a valley to the goal state at the hilltop [25]. The state $\mathbf{s} = [x, \dot{x}]$ of the system is given by the position $x \in [-1.2, 0.6]$ of the car and by its current velocity $\dot{x} \in [-0.07, 0.07]$. Actions are discrete forces applied to the car $a \in \{-a_{\text{max}}, 0, a_{\text{max}}\}$. In every time step a reward of -1 is given to the agent. The initial state is uniformly drawn from $\{(-0.5, \dot{x}) \mid \dot{x} \in [-0.07, 0.07]\}$. For the noisy mountain car [14], zero-mean Gaussian noise with standard deviation 0.01 is added to the observation. This task is now partially observable. A trial is *successful* if the final policy allows the car to reach the hilltop in less than 100 time steps on average in the fully observable case and in less than 120 steps in the partially observable case.

Single- and double pole balancing.

We examine two variants of pole balancing, single-pole balancing as in [22] and double-pole balancing as in [15, 7]. One or two poles are mounted on a 1-dimensional cart and the objective is to balance the poles as long as possible. In the double-pole case both poles are positioned side by side. The state is given by the current position x and velocity $\dot{x}$ of the cart and the current angle ζ (or angles ζ_1, ζ_2) and angular velocity $\dot{\zeta}$ (or velocities $\dot{\zeta}_1, \dot{\zeta}_2$). In the single-pole balancing scenario the agent receives a reward of 0 for every time step the cart is close to the center ($|x| < 0.05$) and the pole is almost perpendicular ($|\zeta| < 0.05$), a reward of $-2(N - t)$ (with maximal episode length $N = 500$) if the pole crashes ($|\zeta| > 0.7$) or leaves the allowed x -range ($|x| > 2.4$) at time step t , and a reward of -1 in every other time step. A trial is regarded as successful if the ac-

cumulated reward of the final policy is larger than -200 (implying that the pole is balanced at least most of the time). The starting point for an episode is drawn from the set $\{(x, 0, \zeta, 0) | x \in [-2, 2], \zeta \in [-0.6, 0.6]\}$. In the double-pole balancing task the reward is 1 for every time step and we regard a trial as successful if the final policy avoids crashing for 1000 time steps. The system state is initialized with $(x, \dot{x}, \zeta_1, \dot{\zeta}_1, \zeta_2, \dot{\zeta}_2) = (0, 0, \mathcal{N}(1^\circ, 1), 0, 0, 0)$.

Swimmer.

The highest dimensional task we consider is the swimmer problem [5]. The swimmer consists of n_c compartments floating on a liquid surface. The swimmer is supposed to move its center of gravity as fast as possible in a predefined direction. The state description

$$\mathbf{s} = [A_0, \zeta_1, \dots, \zeta_{n_c}, \dot{A}_0, \dot{\zeta}_1, \dots, \dot{\zeta}_{n_c}]^T$$

includes the position of the end point of the first compartment A_0 (marking the “head” of the swimmer), the angle of the i th part ($i = 1, \dots, n_c$) with respect to the x -axis, the corresponding velocity of the “head” $\dot{A}_0$, and the angular velocities for each part $\dot{\zeta}_1, \dots, \dot{\zeta}_{n_c}$. Actions $\mathbf{a} = [a_1, \dots, a_{n_c-1}]^T$ are torques applied between body parts. The reward given to the swimmer is the velocity component of the swimmer’s center of gravity parallel to the x -axis. We considered two swimmers with $n_c \in \{3, 4\}$. The swimmer’s initial position is set to 0 and the initial angular velocities are each drawn uniformly from $[0, \pi]$. A swimmer trial is called successful if the average velocity of the swimmer’s center of gravity is larger than $\frac{3n_c}{40} \frac{\text{m}}{\text{s}}$, i.e., the swimmer covers a distance of at least one and a half of its length in the desired direction in the simulated time span of 20 s.

4.2 Baseline algorithms

In order to judge the performance of the CMA-ES and the UH-CMA-ES for RL, we consider two methods for comparison. First, random weight guessing serves as baseline algorithm. Second, we apply the NAC algorithm according to [20], a state-of-the-art policy gradient method.

Random weight guessing.

Stochastic hill-climbing often gives surprising results on standard RL benchmarks. We use simple random search (random weight guessing, RWG) as a baseline for evaluating whether a learning algorithm works or not [7]. In every iteration new policy parameters are drawn uniformly from an interval $[-\theta_{\max}, \theta_{\max}]^k$, where k is the number of policy parameters. This candidate solution is evaluated and is maintained if it outperforms the best solution so far and discarded otherwise.

Natural policy gradient ascent.

Policy gradient methods operate on a predefined class of stochastic policies. They require a differentiable structure to ensure the existence of the gradient of the performance measure and ascent this gradient. Let the performance $\rho(\pi)$ of the current policy with parameters θ be defined as above. Because in general neither d^π , $\mathcal{R}$, nor $\mathcal{P}$ are known, the performance gradient $\nabla_{\theta} \rho(\pi)$ with respect to the policy parameters θ is estimated from interaction with the environment.

The policy gradient theorem with function approximation [26] ensures that an unbiased estimate of the performance gradient can be determined from unbiased estimates of the

stationary distribution and the state-action value function $Q^\pi(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_{t+1} | \pi, s_0 = s, a_0 = a]$ (where $r_{t+1} \in \mathbb{R}$ is the reward received after the action in time step t). For any MDP it holds

$$\nabla_{\theta} \rho(\pi) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi(s, a) f_v^\pi(s, a) .$$

Here a function approximator $f_v : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ (the *critic*) with real-valued parameter vector $\mathbf{v}$ satisfying the *convergence condition*

$$\sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi(s, a) [Q^\pi(s, a) - f_v(s, a)] \nabla_{\mathbf{v}} f_v(s, a) = 0$$

is used instead of the unknown $Q^\pi(s, a)$. The function f_v needs to be *compatible* with the policy parametrization in the sense that $f_v = \nabla_{\theta} \ln(\pi(s, a)) \mathbf{v} + \text{const}$.

Stochastic policies π with parameters θ are parametrized probability distributions. The Fisher information matrix $F(\theta)$ induces a metric in the space of probability distributions that is independent of the coordinate system [1]. The direction of steepest ascent in this metric space is given by $\tilde{\nabla}_{\theta} \rho(\pi) = F(\theta)^{-1} \nabla_{\theta} \rho(\pi)$, thus inducing “natural” gradient ascent in this direction. We have

$$F(\theta) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi(s, a) \nabla_{\theta} \ln(\pi(s, a)) (\nabla_{\theta} \ln(\pi(s, a)))^T$$

using the definitions above. This implies $\nabla_{\theta} \rho(\pi) = F(\theta) \mathbf{v}$, which leads to the simple equality $\tilde{\nabla}_{\theta} \rho(\pi) = \mathbf{v}$.

For episodic tasks it is not necessary to explicitly compute the advantage function $f_v(s, a)$, since in this case the performance gradient can be estimated directly from the Bellman equations (see [20] for details).

4.3 Experimental setup

In all tasks uncertainty arises from random start states. In the POMDP mountain car task the observation is noisy. We always consider the same type of linear policies in order to allow for a fair comparison. The linear policies considered here are typically used with the NAC [22, 21, 20, 12]). More sophisticated choices of policy classes certainly improve the performance of the CMA-ES, which for example works fine with non-linear neural networks [15, 19, 7]. Thus all methods operate on the same policy class $\pi_{\theta}^{\text{deter}}(\mathbf{s}) = \theta^T \mathbf{s}$ with $\mathbf{s}, \theta \in \mathbb{R}^n$. For learning, the NAC uses the stochastic policy $\pi_{\theta}^{\text{stoch}}(\mathbf{s}, a) = \mathcal{N}(\pi_{\theta}^{\text{deter}}(\mathbf{s}), \sigma_{\text{NAC}})$, where the standard deviation σ_{NAC} is considered as an additional adaptive parameter of the PGM. The NAC is evaluated on the corresponding deterministic policy. The policy parameters (except the exploration parameter σ_{NAC} for the NAC) are always initialized with zero. For the swimmer task the action consists of $n_c - 1$ continuous values and we apply an independent linear policy for each action component, thus the search space is $2(n_c + 1)(n_c - 1)$ -dimensional.

We considered various hyperparameter configurations for each algorithm. For the independent evaluation of the algorithms (e.g. in the following plots) we use 50 roll-outs, except for the swimmer task where only 20 roll-outs are used. For the fully observable mountain car problem and the balancing tasks we test for the CMA-ES all combinations of initial global step size $\sigma^{(0)} \in \{0.1, 1, 5, 10, 15, 25, 50, 100\}$ and sample size $n_{\text{eval}} \in \{1, 10, 20, 30\}$, and for the NAC all combinations of learning rate $\alpha_{\text{NAC}} \in \{0.1, 0.01, 0.001, 10^{-4}\}$, initial exploration $\sigma_{\text{NAC}} \in \{0.1, 1, 5, 10, 15, 25, 50, 100\}$, and

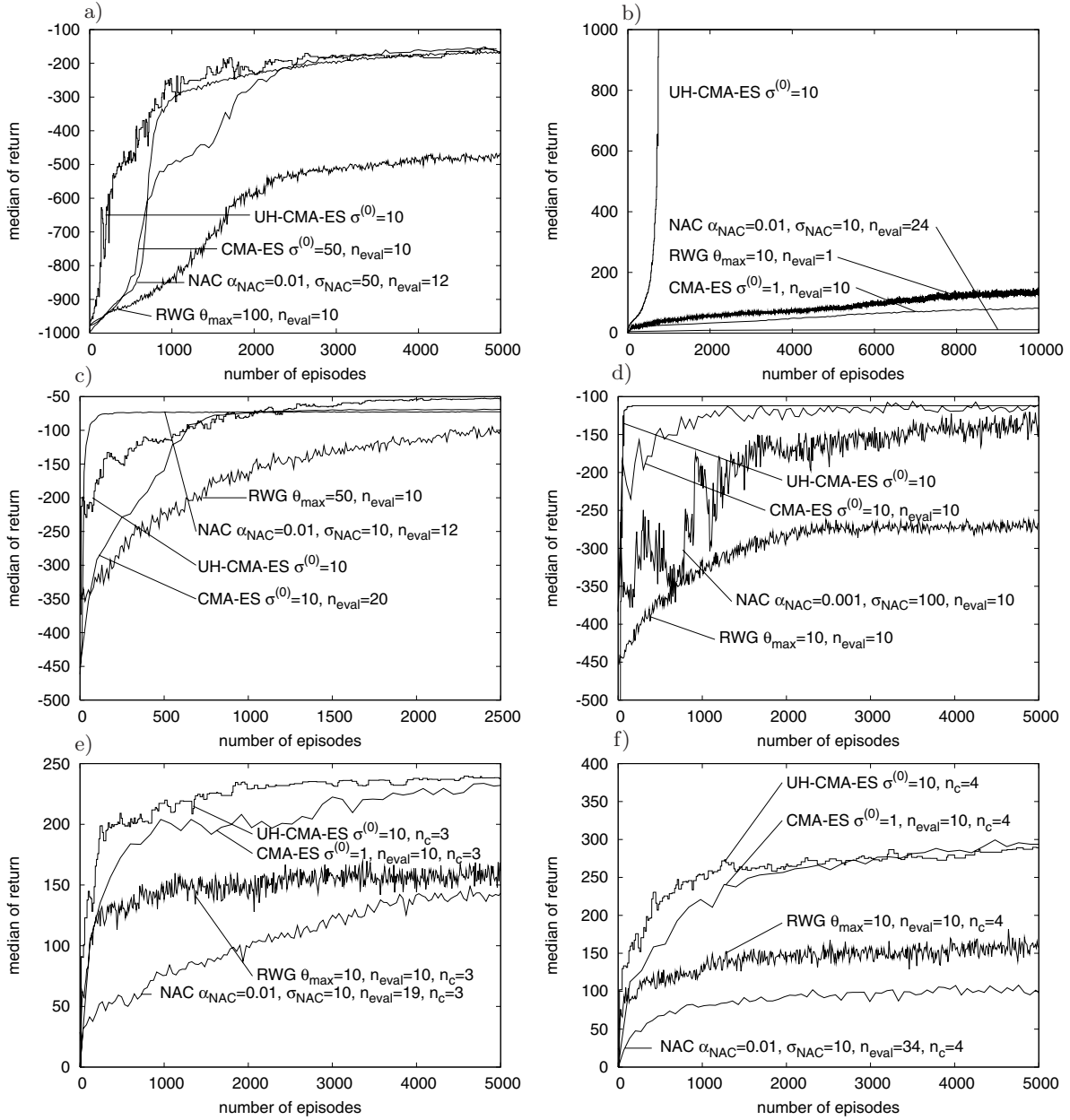


Figure 1: Median of performance over 500 trials for NAC, RWG, CMA-ES, and UH-CMA-ES for a) single-pole balancing, b) double-pole balancing, c) mountain car task, and d) partially observable mountain car task. For each method, the results for the best hyperparameter configuration are shown. For the swimmers with 3 and 4 segments the median of 20 trials is plotted in e) and f), respectively.

sample size $n_{\text{eval}} \in \{n+2, 2(n+2), 3(n+2)\}$ (the NAC needs a minimum of $n+2$ roll outs per policy update). For the UH-CMA-ES we vary $\sigma^{(0)} \in \{0.1, 1, 5, 10, 15, 25, 50, 100\}$ and set $n_{\text{eval}} = 1$. For the partially observable mountain car problem we use the rank-1 CMA-ES and test $\sigma^{(0)} \in \{1, 10, 100\}$ with $n_{\text{eval}} = 10$, and for the NAC we use $\sigma_{\text{NAC}} \in \{1, 10, 50, 100\}$ with $n_{\text{eval}} = 10$. For the UH-CMA-ES we consider the same configurations as in the fully observable task. Since the swimmer problem is the most computational demanding we restricted $\sigma^{(0)}$ and σ_{NAC} to $\{1, 10, 50\}$ for this task. For RWG we test for all experiments all combinations

of interval length $\theta_{\text{max}} \in \{5, 10, 20, 30, 40, 50, 100\}$ and sample size $n_{\text{eval}} \in \{1, 10, 20, 30\}$.

4.4 Results

In Figure 1 the performances of CMA-ES, NAC, RWG and UH-CMA-ES are shown for the six test scenarios. For all three methods the performance of the best respective hyperparameter configuration is plotted. Both the NAC and the CMA-ES benefit from larger sample sizes n_{eval} . As can be seen in Figure 2a) for single-pole balancing, too small sample sizes result in fast learning in the beginning but lead

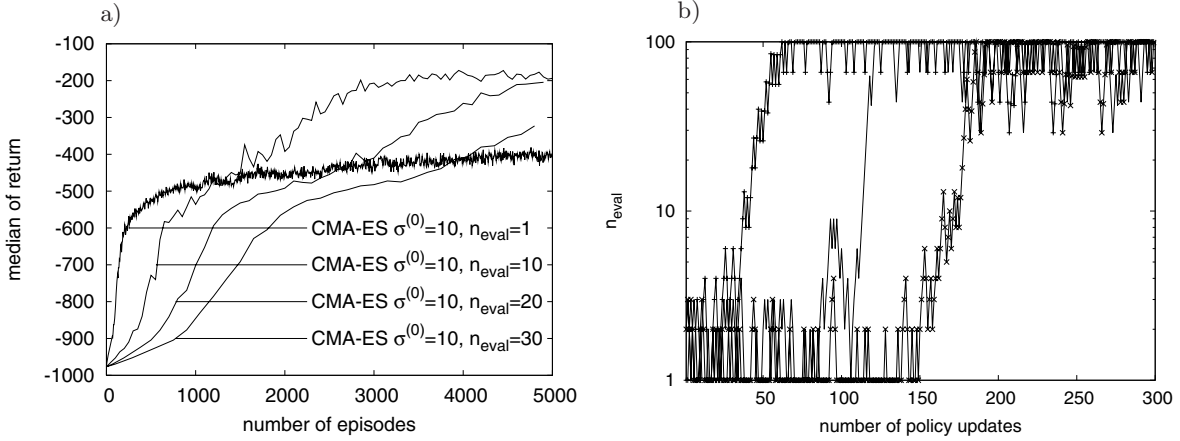


Figure 2: a) Performance of the CMA-ES for different sample sizes $n_{\text{eval}} \in \{1, 10, 20, 30\}$ in the single-pole balancing experiment. b) Sample sizes computed by the UH-CMA-ES for three typical single-pole balancing trials.

to early stagnation (i.e., the uncertainty is too high to make progress). On the other hand too large sample sizes allow the CMA-ES to find much better solutions but lead to unnecessarily slow learning. In Figure 2b) the sample sizes chosen by the UH-CMA-ES are shown for typical single trials. At the beginning small sample sizes are used and only when the accuracy is no longer sufficient the sample size is increased. Finally for fine tuning the maximal allowed sample size of $n_{\text{eval}} = 100$ is used most of the time. Altogether the uncertainty handling clearly improves the learning speed, while the CMA-ES and the NAC without uncertainty handling perform roughly equally well when looking at the best respective hyperparameter configurations. Only in the fully observable mountain car problem the UH-CMA-ES is not faster than the NAC and the CMA-ES, see Figure 1c). Here the NAC clearly outperforms both CMA-ES versions, since the NAC benefits from the policy parameter initialization close to a global optimum [14]. The uncertainty handling improves the CMA-ES's results but suffers from discontinuities of the accumulated reward originating in the large initial state interval that includes initial velocities almost sufficient to drive the car directly uphill. But in the more challenging partially observable task the UH-CMA-ES strikingly outperforms all other methods, see Figure 1d).

But the uncertainty handling does not only improve the learning speed. For the fully observable mountain car task the UH-CMA-ES is successful in more than 80% of the 500 trials for 6 out of 8 tested hyperparameter configurations, while the standard CMA is successful in more than 80% of the trials in only 18 of 32 cases, the NAC in 60 of 96 cases and RWG in 7 of 28 cases. For the mountain car POMDP the UH-CMA-ES is successful in 50% of the trials for 5 of 6 hyperparameter configurations, while CMA-ES, NAC and RWG achieve a success level of 50% in 3 of 3, 6 of 12 and 0 of 28 cases, respectively. (In this case only the most effective fixed sample size was tested.) The same effect can be seen in the two pole balancing tasks. In the single-pole balancing task the UH-CMA-ES is in more than 50% of the trials successful for 4 of 8 hyperparameter configurations, while the CMA-ES and the NAC achieve the same success level with 21 of 32 and with 7 of 96 hyperparameter configurations, respectively.

The RWG reaches this success level in 4 of 28 hyperparameter configurations. For the double-pole balancing task (which is even without noise a difficult problem when considering only linear policies [12]) the results are most striking. Here the UH-CMA-ES is successful in 100% of the trials for all 8 tested hyperparameter configurations (which is even better than the CMA-ES for a fixed start state) while the CMA-ES and the NAC only achieve a success level of 5% with 5 of 32, 27 of 96 hyperparameter configurations, respectively and are both more robust than RWG with 2 of 28 successful configurations. For swimmers both with 3 and 4 compartments the UH-CMA-ES is also robust. The UH-CMA-ES is successful in more than 50% of all trials for 3 of 3 and 1 of 3 hyperparameter configurations for $n_c \in \{3, 4\}$, respectively. The CMA-ES achieves success in more than 50% of the trials in 11 of 12 and 5 of 12 cases for $n_c \in \{3, 4\}$, while the NAC and RWG reach this success level in 0 of 36 and 0 of 7 cases for $n_c = 3$ and 0 of 36 and 0 of 7 cases for $n_c = 4$. Thus the rank based uncertainty handling remarkably increases the learning speed while at the same time improving the robustness compared to the CMA-ES without uncertainty handling, which is already much more robust than the PGM.

5. CONCLUSION

Evolution strategies (ES) are powerful direct policy search methods. One of their main advantages is their ability to cope with noise. Still, random elements in the environment require gathering statistics over several episodes for the evaluation of candidate policies. We added an adaptive uncertainty handling developed by Hansen et al. [9, 10] to evolutionary reinforcement learning. It adjusts the number of roll-outs per evaluation of a policy such that the signal to noise ratio is just high enough for a sufficiently good ranking of candidate policies, which in turn suffices for the ES to find better solutions. The uncertainty handling exploits the advantage of small sample sizes in the beginning and only increases the sample size when a higher accuracy is necessary thus balancing the trade-off between fast learning and sufficient accuracy. This significantly increases both learning speed and robustness and allows to find highly accurate

final solutions. It was in particular highly effective in the partially observable benchmark problem with noisy observations.

In future work, we will extend our approach to allow for an unequal distribution of evaluations among the offspring in one generation.

6. ACKNOWLEDGMENTS

The authors acknowledge support from the German Federal Ministry of Education and Research within the Bernstein group “The grounding of higher brain function in dynamic neural fields”.

7. REFERENCES

- [1] S. Amari and H. Nagaoka. *Methods of Information Geometry*. Number 191 in Translations of Mathematical Monographs. American Mathematical Society and Oxford University Press, 2000.
- [2] D. V. Arnold. *Noisy Optimization With Evolution Strategies*. Kluwer Academic Publishers, 2002.
- [3] H.-G. Beyer. Evolution strategies. *Scholarpedia*, 2(8):1965, 2007.
- [4] C. Chen and E. Yucesan. An alternative simulation budget allocation scheme for efficient simulation. *International Journal of Simulation and Process Modeling*, 1(1):49–57, 2005.
- [5] R. Coulom. Apprentissage par renforcement utilisant des reseaux de neurones, avec des applications au controle moteur. *These de doctorat, Institut National Polytechnique de Grenoble*, 2002.
- [6] T. Glasmachers and C. Igel. Uncertainty handling in model selection for support vector machines. In G. Rudolph, editor, *Parallel Problem Solving from Nature (PPSN X)*, volume 5199 of *LNCS*, pages 185–194. Springer-Verlag, 2008.
- [7] F. Gomez, J. Schmidhuber, and R. Miikkulainen. Accelerated neural evolution through cooperatively coevolved synapses. *Journal of Machine Learning Research*, 9:937–965, 2008.
- [8] N. Hansen, S. D. Müller, and P. Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003.
- [9] N. Hansen, A. S. P. Niederberger, L. Guzzella, and P. Koumoutsakos. Evolutionary optimization of feedback controllers for thermoacoustic instabilities. In J. F. Morrison, D. M. Birch, and P. Lavoie, editors, *IUTAM Symposium on Flow Control and MEMS*. Springer-Verlag, 2008.
- [10] N. Hansen, A. S. P. Niederberger, L. Guzzella, and P. Koumoutsakos. A method for handling uncertainty in evolutionary optimization with an application to feedback control of combustion. *IEEE Transactions on Evolutionary Computation*, 13(1):180–197, 2009.
- [11] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.
- [12] V. Heidrich-Meisner and C. Igel. Evolution strategies for direct policy search. In G. Rudolph, editor, *Parallel Problem Solving from Nature (PPSN X)*, number 5199 in *LNCS*, pages 428–437. Springer-Verlag, 2008.
- [13] V. Heidrich-Meisner and C. Igel. Uncertainty handling in evolutionary direct policy search. In Y. Engel, M. Ghavamzadeh, P. Poupart, and S. Mannor, editors, *NIPS-08 Workshop on Model Uncertainty and Risk in Reinforcement Learning*. 2008.
- [14] V. Heidrich-Meisner and C. Igel. Variable metric reinforcement learning methods applied to the noisy mountain car problem. In S. Girgin et al., editors, *European Workshop on Reinforcement Learning (EWRL 2008)*, number 5323 in *LNAI*, pages 136–150. Springer-Verlag, 2008.
- [15] C. Igel. Neuroevolution for reinforcement learning using evolution strategies. In *Congress on Evolutionary Computation (CEC 2003)*, volume 4, pages 2588–2595. IEEE Press, 2003.
- [16] C. Igel, T. Glasmachers, and V. Heidrich-Meisner. Shark. *Journal of Machine Learning Research*, 9:993–996, 2008.
- [17] L. Kaelbling, M. Littman, and A. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [18] S. Kakade. A natural policy gradient. In T. G. Dietterich, S. Becker, and Z. Ghahramani, editors, *Advances in Neural Information Processing Systems (NIPS14)*. MIT Press, 2002.
- [19] A. Pellecchia, C. Igel, J. Edelbrunner, and G. Schöner. Making driver modeling attractive. *IEEE Intelligent Systems*, 20(2):8–12, 2005.
- [20] J. Peters and S. Schaal. Natural actor-critic. *Neurocomputing*, 71(7-9):1180–1190, 2008.
- [21] J. Peters, S. Vijayakumar, and S. Schaal. Reinforcement learning for humanoid robotics. In *Proc. 3rd IEEE-RAS Int’l Conf. on Humanoid Robots*, pages 29–30, 2003.
- [22] M. Riedmiller, J. Peters, and S. Schaal. Evaluation of policy gradient methods and variants on the cart-pole benchmark. In *Proc. IEEE Int’l Symposium on Approximate Dynamic Programming and Reinforcement Learning (ADPRL 2007)*, pages 254–261, 2007.
- [23] C. Schmidt, J. Branke, and S. Chick. Integrating techniques from statistical ranking into evolutionary algorithms. In *Applications of Evolutionary Computing*, volume 3907 of *LNCS*, pages 752–763. Springer, 2006.
- [24] N. T. Siebel and G. Sommer. Evolutionary reinforcement learning of artificial neural networks. *International Journal of Hybrid Intelligent Systems*, 4(3):171–183, 2007.
- [25] R. Sutton and A. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [26] R. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, pages 1057–1063, 2000.
- [27] T. Sutorp, N. Hansen, and C. Igel. Efficient covariance matrix update for variable metric evolution strategies. *Machine Learning*, 75(2):167–197, 2009.
- [28] S. Whiteson and P. Stone. Evolutionary function approximation for reinforcement learning. *Journal of Machine Learning Research*, 7:877–917, 2006.